REQUISITOS FUNCIONALES Y ENDPOINTS

Proyecto Peluquería – API REST (Node.js + Express + MySQL). Actualizado el 21/10/2026.

REQUISITOS FUNCIONALES

- **RF - 01:** Admin crea un cliente
- **RF - 02:** Cliente crea su propia cuenta
- **RF - 03:** Cliente inicia sesión
- **RF - 04:** Admin crea trabajador
- **RF - 05:** Trabajador inicia sesión
- **RF - 06:** Cliente ve horarios
- **RF - 07:** Admin ve horarios
- **RF - 08.1:** Trabajador ve sus horarios (por fecha)
- **RF - 08.2:** Trabajador ve sus horarios (por fecha y estado)
- **RF - 08.3:** Trabajador ve sus horarios (por ID)
- **RF - 09:** Trabajador edita horario
- **RF - 10:** Trabajador elimina horario
- **RF - 11:** Admin edita horario
- **RF - 12:** Admin elimina horario
- **RF - 13:** Admin elimina trabajador
- **RF - 14:** Admin elimina cliente
- **RF - 15.1:** Cliente cancela una cita
- **RF - 15.2:** Cliente reprograma una cita
- **RF - 16:** Admin crea Admin
- **RF - 17:** Admin inicia sesión
- **RF - 18:** Admin activa cliente
- **RF - 19:** Admin desactiva cliente
- **RF - 20:** Admin activa trabajador
- **RF - 21:** Admin desactiva trabajador
- **RF - 22:** Cliente desactiva su cuenta
- **RF - 23:** Admin desactiva admin
- **RF - 24:** Admin activa admin
- **RF - 25:** Admin elimina admin
- **RF - 26:** Cliente crea cita

DATOS GENERALES DE LA API

Puerto: 5020 (variable PORT del archivo .env; si no existe, usa 3333).

Formato: JSON (Content-Type: application/json). Métodos permitidos por CORS: GET, POST, PUT y DELETE.

Campos t1 a t7 en las creaciones: t1 tipoDocumento (CC, TI, CE), t2 numeroDocumento (8 a 10 dígitos), t3 nombres, t4 dirección, t5 teléfono (10 dígitos), t6 correo, t7 contraseña (mínimo 8 caracteres con mayúscula, minúscula, número y símbolo).

Roles: la tabla trabajadores guarda trabajadores (rol "trabajador") y administradores (rol "admin"). Los estados de cuenta son "Activo" e "Inactivo".

Seguridad: las contraseñas se guardan con bcrypt. Todavía no hay token ni control de roles por ruta; está pendiente.

LISTA DE ENDPOINTS (PARTE 1)

RF	Nombre	Endpoint	Datos de entrada	Códigos	Estado
RF - 01	Admin crea un cliente	`POST /seguridad/crearcliente`	t1 a t7	201, 400, 409, 500	Implementado
RF - 02	Cliente crea cuenta	`POST /usuario/crear`	t1 a t7	201, 400, 409, 500	Implementado
RF - 03	Cliente inicia sesión	`POST /cliente/login`	t1 correo, t2 contraseña	200, 400, 401, 403, 500	Implementado
RF - 04	Admin crea trabajador	`POST /seguridad/creartrabajador`	t1 a t7	201, 400, 409, 500	Implementado
RF - 05	Trabajador inicia sesión	`POST /trabajador/login`	t1 correo, t2 contraseña	200, 400, 401, 403, 500	Implementado
RF - 06	Cliente ve horarios	`GET /horarios`	Ninguna	200, 500	Implementado
RF - 07	Admin ve horarios	`GET /seguridad/horarios/:idtrabajador`	:idtrabajador (numérico)	200, 500	Implementado
RF - 08.1	Trabajador ve horarios	`GET /trabajador/horariod/:id&:fecha`	:id, :fecha	200, 400, 500	Implementado
RF - 08.2	Trabajador ve horarios	`GET /trabajador/horarioe/:id&:fecha&:est`	:id, :fecha, :est	200, 400, 500	Implementado
RF - 08.3	Trabajador ve horarios	`GET /trabajador/horariot/:id`	:id (numérico)	200, 400, 500	Implementado
RF - 09	Trabajador edita horario	`PUT /trabajador/horarios/:idhorario`	fecha (AAAA-MM-DD), horaInicio, horaFin	200, 400, 404, 409, 500	Implementado
RF - 10	Trabajador elimina horario	`DELETE /trabajador/horarios/:idhorario`	:idhorario	200, 404, 409, 500	Implementado
RF - 11	Admin edita horario	`PUT /seguridad/horarios/:idhorario`	fecha, horaInicio, horaFin	200, 400, 404, 409, 500	Implementado
RF - 12	Admin elimina horario	`DELETE /seguridad/horarios/:idhorario`	:idhorario (numérico)	200, 400, 404, 409, 500	Implementado

LISTA DE ENDPOINTS (PARTE 2)

RF	Nombre	Endpoint	Datos de entrada	Códigos	Estado
RF - 13	Admin elimina trabajador	`DELETE /seguridad/trabajadores/:idtrabajador`	:idtrabajador (numérico)	200, 400, 403, 404, 409, 500	Implementado
RF - 14	Admin elimina cliente	`DELETE /seguridad/clientes/:idcliente`	:idcliente (numérico)	200, 400, 404, 409, 500	Implementado
RF - 15.1	Cliente cancela cita	`PUT /citas/:idcita/cancelar`	idcliente (body)	200, 400, 403, 404, 409, 500	Implementado
RF - 15.2	Cliente reprograma cita	`PUT /citas/:idcita/reprogramar`	idcliente, idhorario (body)	200, 400, 403, 404, 409, 500	Implementado
RF - 16	Admin crea Admin	`POST /seguridad/crearadmin`	t1 a t7	201, 400, 409, 500	Implementado
RF - 17	Admin inicia sesión	`POST /seguridad/login`	t1 correo, t2 contraseña	200, 400, 401, 403, 500	Implementado
RF - 18	Admin activa cliente	`PUT /seguridad/activarcliente/:id`	:id (numérico)	200, 400, 404, 500	Implementado
RF - 19	Admin desactiva cliente	`PUT /seguridad/desactivarcliente/:id`	:id (numérico)	200, 400, 404, 500	Implementado
RF - 20	Admin activa trabajador	`PUT /seguridad/activartrabajador/:id`	:id (numérico)	200, 400, 404, 500	Implementado
RF - 21	Admin desactiva trabajador	`PUT /seguridad/desactivartrabajador/:id`	:id (numérico)	200, 400, 404, 500	Implementado
RF - 22	Cliente desactiva cuenta	`PUT /usuario/desactivar/:idcliente`	:idcliente (numérico)	200, 400, 404, 409, 500	Implementado
RF - 23	Admin desactiva admin	`PUT /seguridad/desactivaradmin/:id`	:id (numérico)	200, 400, 404, 409, 500	Implementado
RF - 24	Admin activa admin	`PUT /seguridad/activaradmin/:id`	:id (numérico)	200, 400, 404, 500	Implementado
RF - 25	Admin elimina admin	`DELETE /seguridad/admins/:idadmin`	:idadmin (numérico)	200, 400, 403, 404, 409, 500	Implementado
RF - 26	Cliente crea cita	`POST /citas`	Datos de la cita (body)	201, 400, 500	Implementado

OBSERVACIONES

1. El RF-07 incluye la consulta por trabajador (`GET /seguridad/horarios/:idtrabajador`).
2. Cancelar una cita la deja en estado CANCELADA y libera el horario (DISPONIBLE). Reprogramar solo permite un horario DISPONIBLE del mismo trabajador.